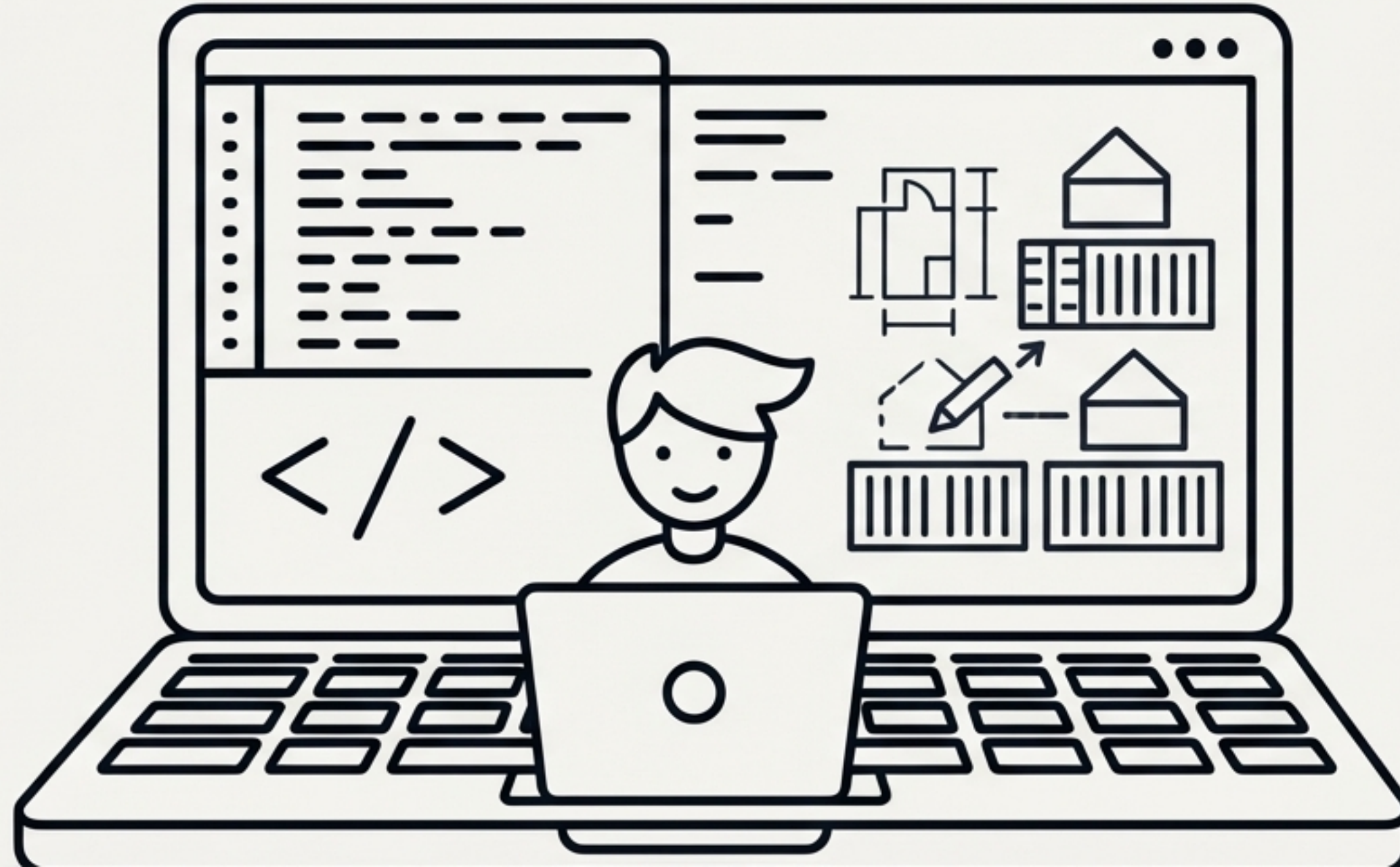


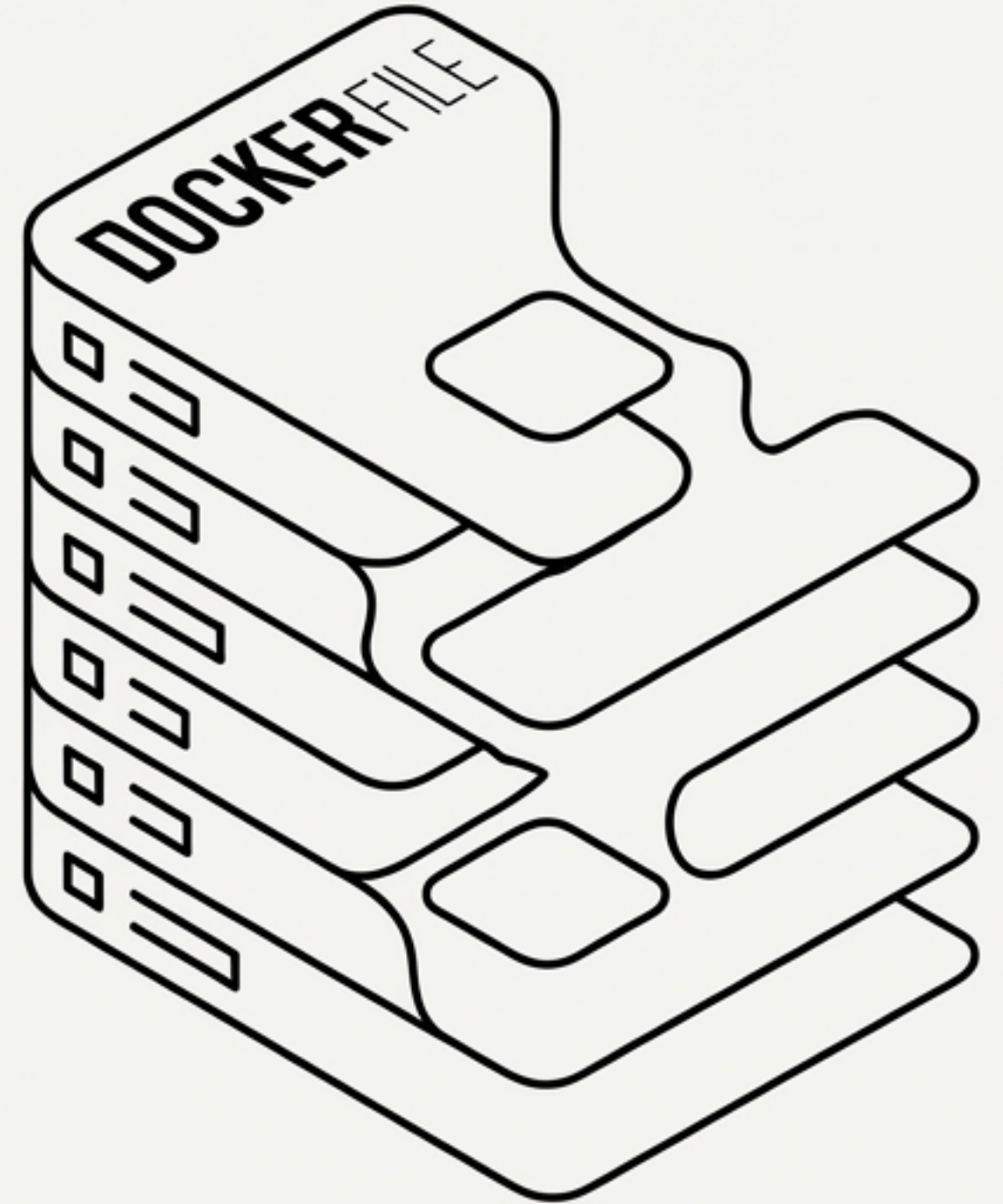
Construção de Imagens com Dockerfile: O Guia do Arquiteto

Domine a arte de criar imagens Docker otimizadas, do alicerce às **técnicas avançadas** para builds eficientes e seguros em produção.



O Dockerfile é a Receita da Sua Imagem

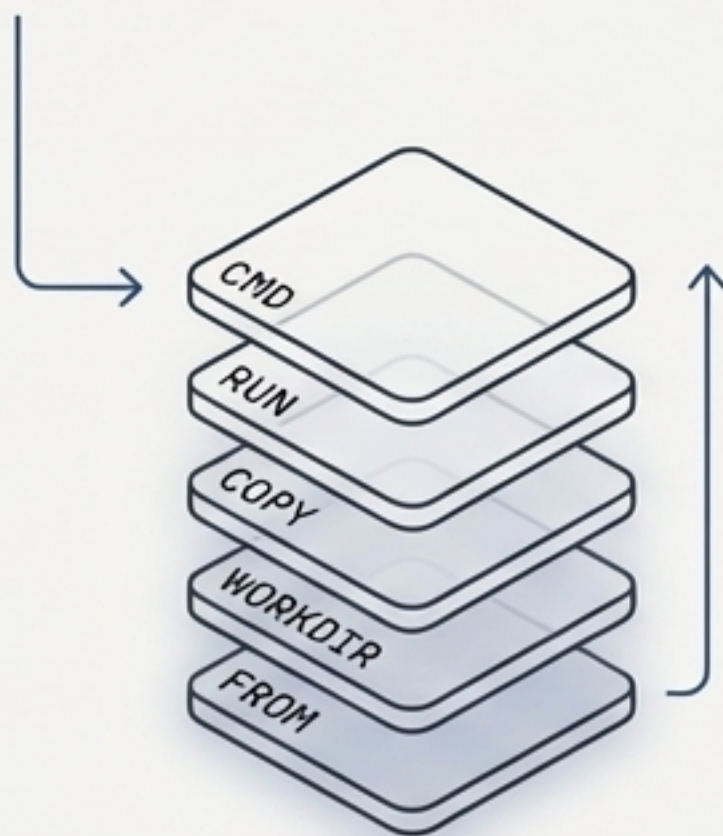
- Um Dockerfile é um arquivo de texto simples contendo uma sequência de instruções que o Docker Engine executa para construir uma imagem.
- Cada instrução cria uma nova camada (layer) na imagem final. Este conceito é a chave para a otimização.
- O comando `docker build` lê este "manual" e o transforma em uma imagem portátil e reutilizável.



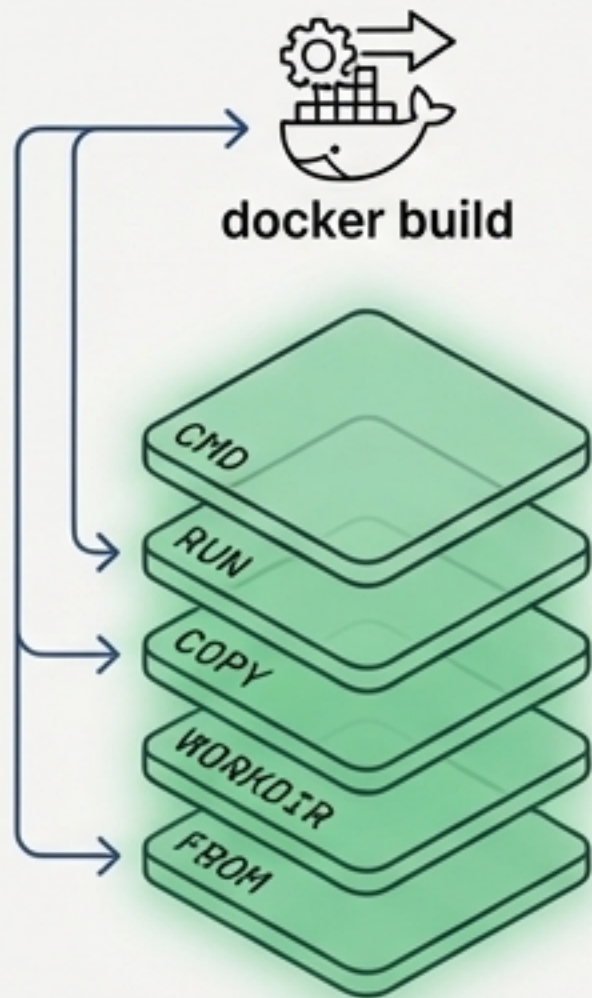
A Física do Build: Camadas Imutáveis e Cache Inteligente

Camadas Imutáveis (Layers)

Cada instrução (`FROM`, `RUN`, `COPY`) gera uma camada de somente leitura. As camadas são empilhadas para formar a imagem final, como blocos de construção.



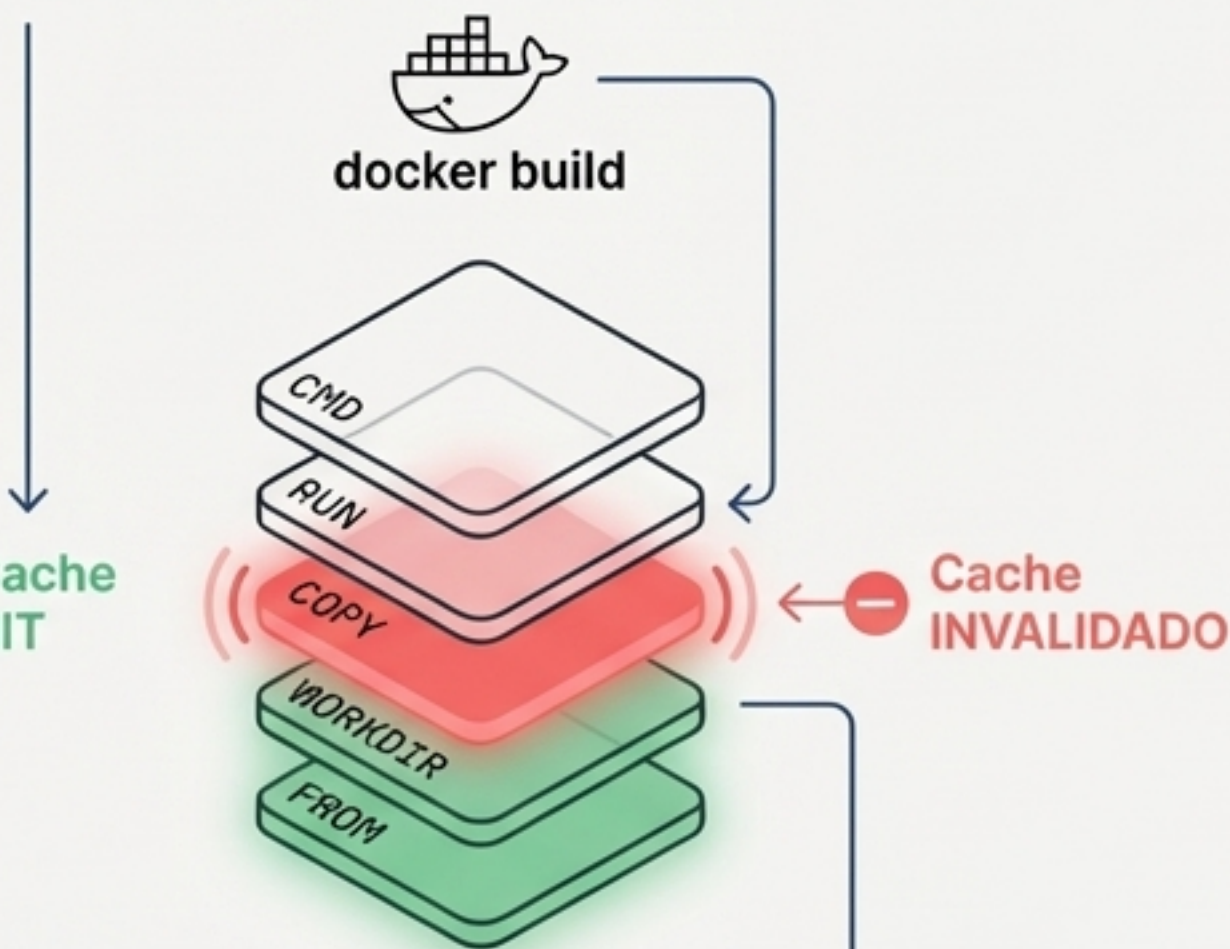
State 1
(Build)



State 2
(Rebuild - No Change)

Cache Inteligente

O Docker reutiliza camadas não modificadas entre builds, acelerando drasticamente o processo.



State 3
(Rebuild - With Change)

A Ordem Importa

A sequência das instruções é crucial. Uma alteração em uma linha invalida o cache para todas as camadas subsequentes.

O Kit de Ferramentas Essencial (Parte I)

FROM

``FROM [imagem:tag]``

Define a imagem base sobre a qual você construirá. Toda imagem Docker parte de outra imagem.

`FROM node:18-alpine`

WORKDIR

``WORKDIR [caminho]``

Estabelece o diretório de trabalho padrão dentro do contêiner para as instruções seguintes.

`WORKDIR /app`

COPY

``COPY [origem] [destino]``

Copia arquivos e diretórios do seu host para o sistema de arquivos da imagem.

`COPY package*.json ./`

O Kit de Ferramentas Essencial (Parte II)

RUN

``RUN [comando]``

Executa comandos durante o build. Usado para instalar dependências e configurar o ambiente. Cada ``RUN`` cria uma nova camada.

```
RUN npm install --production
```

EXPOSE

``EXPOSE [porta]``

Documenta qual porta a aplicação escutará. Não publica a porta automaticamente.

```
EXPOSE 3000
```

CMD

``CMD ["comando"]``

Define o comando padrão que é executado quando o contêiner é iniciado. Pode ser sobrescrito facilmente.

```
CMD ["node", "server.js"]
```

A Arte da Otimização I: Reduzindo Camadas

A estratégia de reduzir o número de camadas acelera o build e diminui o tamanho final da imagem.

Abordagem Ineficiente ✖

```
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y git
RUN rm -rf /var/lib/apt/lists/*
```

— Cria 4 camadas desnecessárias.

Abordagem Otimizada ✔

```
RUN apt-get update && \
    apt-get install -y curl git && \
    rm -rf /var/lib/apt/lists/*
```

— Uma única camada otimizada.

💡 **Dica Pro:** Sempre limpe caches de gerenciadores de pacotes (apt, yum, apk) na mesma instrução `RUN` para não armazenar arquivos temporários na camada.

A Arte da Otimização II: O Poder do `.dockerignore`

O que é?

Similar ao `.gitignore`, este arquivo lista padrões de arquivos e diretórios que não devem ser enviados para o contexto de build do Docker.

Benefícios Principais



- **Acelera o build:** Reduz drasticamente o tamanho do contexto enviado ao Docker daemon.



- **Aumenta a segurança:** Evita o vazamento de informações sensíveis (como credenciais ou chaves `.env`) para a imagem.

Exemplo de `.dockerignore`

```
# Controle de versão
.git
.gitignore

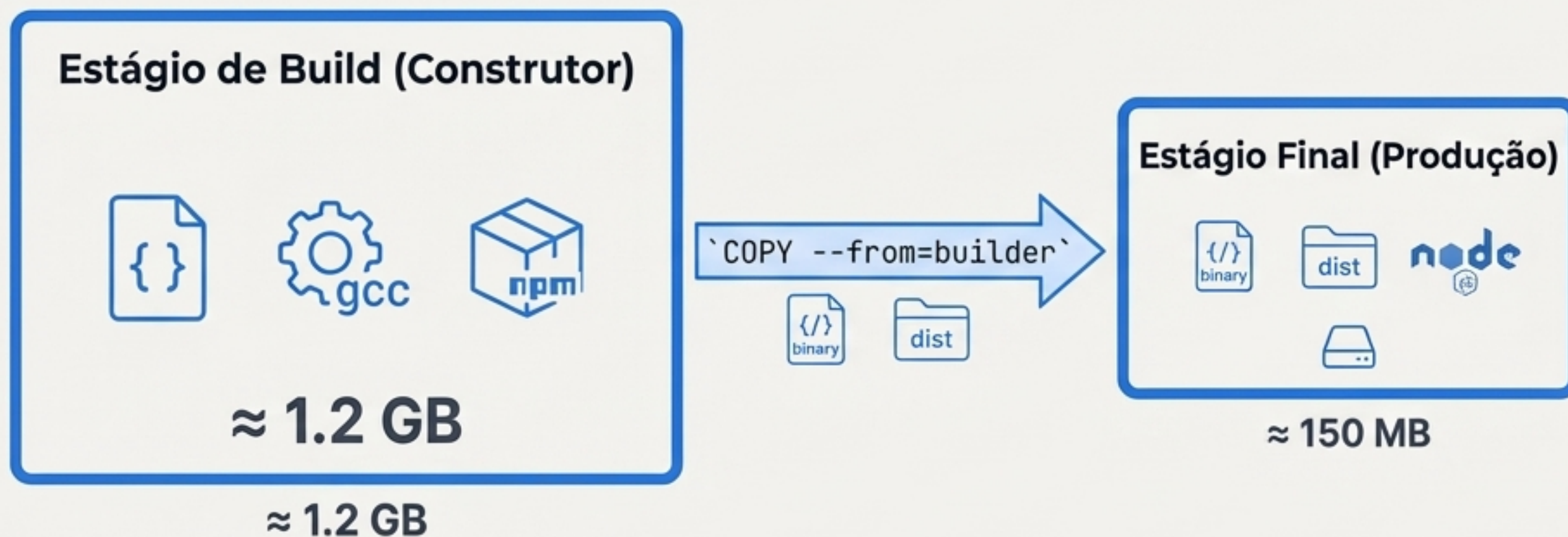
# Dependências
node_modules
vendor

# Arquivos de desenvolvimento
*.md
.env
.env.local

# Logs e temporários
*.log
tmp/
.DS_Store
```

O Toque de Mestre: Multi-Stage Builds

Multi-Stage Builds permitem usar múltiplos estágios `FROM` no mesmo Dockerfile, copiando apenas os artefatos necessários para a imagem final. O resultado são imagens drasticamente menores e mais seguras.



1 Estágio de Build (Construtor)

Contém todas as ferramentas de compilação, dependências de desenvolvimento e o código-fonte. Pode ser pesado.

2 Copiar Artefatos

Usa ``COPY --from=builder`` para transferir apenas os arquivos compilados e essenciais do estágio anterior.

3 Estágio Final (Produção)

Uma imagem mínima, contendo apenas o runtime e os artefatos da aplicação. Leve, rápida e segura.

Multi-Stage na Prática: A Diferença é Drástica

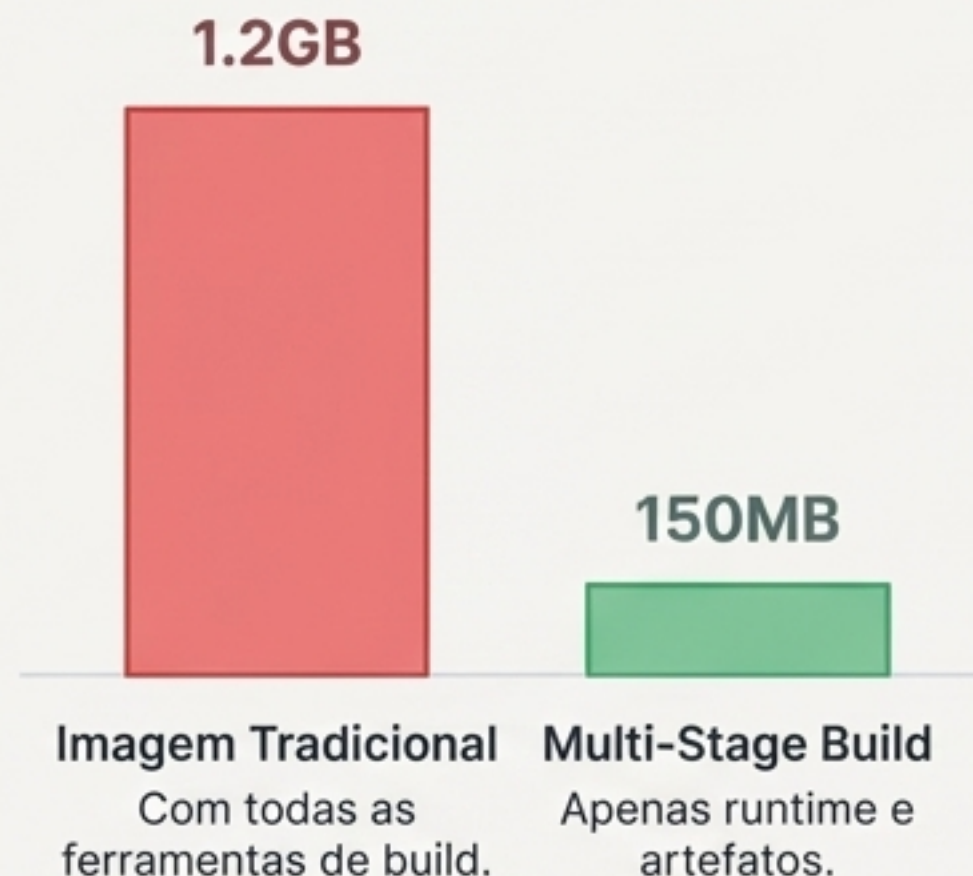
```
# Estágio 1: Build
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
```

Estágio Construtor

```
# Estágio 2: Produção
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Estágio Final

Copia apenas os artefatos



87%
REDUÇÃO

Economia de espaço e tempo

Engenharia de Precisão: CMD vs. ENTRYPOINT

Ambas as instruções definem o que executar quando o contêiner inicia, mas com propósitos e comportamentos distintos.

Aspecto	CMD	ENTRYPOINT
Propósito	Define comando padrão ou argumentos	Define o executável principal
Sobrescrita	Facilmente sobrescrito ao executar	Difícil de sobrescrever (requer --entrypoint)
Uso Comum	Argumentos flexíveis	Executável fixo
Formato	<code>`CMD ["executável", "arg1"]`</code>	<code>`ENTRYPOINT ["executável"]`</code>

A Combinação Ideal: ENTRYPOINT + CMD

Usar `ENTRYPOINT` para o executável fixo e `CMD` para argumentos padrão cria contêineres que são ao mesmo tempo robustos e configuráveis. Esta é a melhor prática para aplicações em produção.

Exemplo Prático



Execução Padrão

`docker run myapp` → Executa `python app.py --port 8080`

Sobrescrevendo Argumentos

`docker run myapp --port 3000` → Executa `python app.py --port 3000`

O Modelo da Excelência: Checklist de Otimização



Use imagens base oficiais e leves: Prefira variantes ``alpine`` ou ``slim``. (Ex: ``node:18-alpine``, ``python:3.11-slim``)



Ordene instruções por frequência de mudança: Coloque comandos que mudam raramente (instalação de dependências) antes daqueles que mudam sempre (cópia de código-fonte) para maximizar o uso do cache.



Minimize o número de camadas: Combine comandos ``RUN`` relacionados usando ``&&`` e quebre linhas com ``\``.



Sempre use ``.dockerignore``: Exclua arquivos desnecessários para acelerar builds e proteger informações sensíveis.



Aplique Multi-Stage Builds: Separe os ambientes de build e produção para gerar imagens finais menores e mais seguras.

Exemplo Completo: Aplicação Node.js Otimizada

Este Dockerfile incorpora todas as técnicas de otimização: imagem Alpine, Multi-Stage Build, cache de dependências, usuário não-root e a combinação `ENTRYPOINT` + `CMD`.

```
# Multi-stage build otimizado
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
# Usa 'npm ci' para builds determinísticos
RUN npm ci --only=production

FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .

EXPOSE 3000
# Prática de segurança: executa como usuário não-root
USER node

ENTRYPOINT ["node"]
CMD ["server.js"]
```

Comandos Essenciais para o Dia a Dia

1. Construir Imagem



```
docker build -t minha-app:v1.0  
.
```

Cria uma imagem a partir do Dockerfile no diretório atual, aplicando a tag `minha-app:v1.0`.

2. Verificar Camadas



```
docker history minha-app:v1.0
```

Inspecciona todas as camadas da imagem, seus tamanhos e os comandos que as criaram.

3. Limpar Cache



```
docker build --no-cache -t app  
.
```

Força um rebuild completo da imagem, ignorando completamente o cache de camadas.

Próximos Passos: O Desafio da Persistência de Dados

Preview da Próxima Aula

Na próxima aula, vamos resolver o desafio dos dados **efêmeros** em contêineres. Exploraremos:

1. **Volumes:** Gerenciados pelo Docker, persistentes e compartilháveis.
2. **Bind Mounts:** Mapeamento direto de diretórios do host.
3. **Casos de Uso:** Quando usar cada estratégia de persistência.



Tarefa Final

Crie um Dockerfile otimizado com Multi-Stage Build para a sua própria aplicação.